Tema 7: Utilización de mecanismos de comunicación asíncrona

Autor: Roberto Marín

Asignatura: Desarrollo Web Entorno Cliente (DAW2)

Año: 2023/2024

COPYRIGHT:

Reconocimiento-NoComercial-CompartirIgual
CC BY-NC-SA

Esta licencia permite a otros entremezclar, ajustar y construir a partir de su obra con fines no comerciales, siempre y cuando le reconozcan la autoría y sus nuevas creaciones estén bajo una licencia con los mismos términos.

No se permite el uso de este material en ninguna convocatoria oficial de oposiciones.

Índice:

Tema 7: Utilización de mecanismos de comunicación asíncrona 1

7.1. Comunicación Asíncrona con AJAX 3

7.1.1 Introducción a Ajax 3

7.1.2 Promesas: Async y Await 5

7.1.3 API, Métodos y Propiedades del objeto XMLHttpRequest: 5

7.1.4 Ajax y Json 8

7.2. Bibliografía 10

7.3. Agradecimientos y reconocimiento 10

7.1. Comunicación Asíncrona con AJAX

7.1.1 Introducción a Ajax

La palabra AJAX viene de la conjunción de JavaScript Asíncrono y XML. Sirve como capa intermedia entre el servidor y el navegador, y se emplea Javascript para conectar las diferentes tecnologías.

Nos permite actualizar nuestra aplicación Web sin necesidad de recargar la pantalla, debido a que Ajax trabaja en un segundo plano.

Y además, se podrá recibir o enviar información a nuestro servidor.

AJAX está compuesto por las siguientes tecnologías:

HTML y CSS, para la capa de presentación de la aplicación.

DOM, para la interacción y manipulación dinámica de la presentación.

Gestión de datos: **XML** (mensajes complejos y tipificados), **JSON** (menor tamaño fichero para móviles), otros tipos de ficheros.

Recuperación de datos de manera asíncrona (OldSchool): usando el objeto **XMLHttpRequest**.

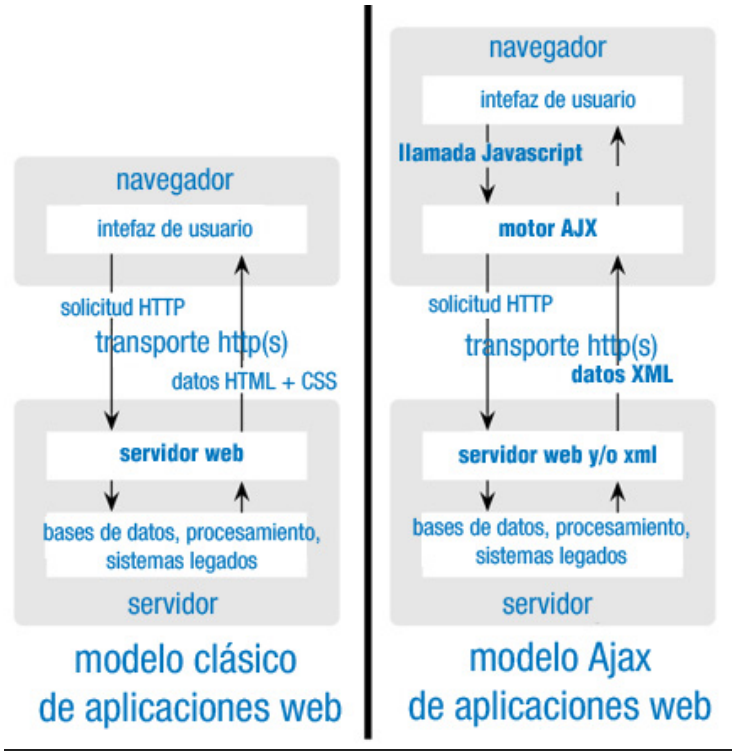
Recuperación de datos de manera asíncrona basada en promesas (NewSchool): **Fetch API.**

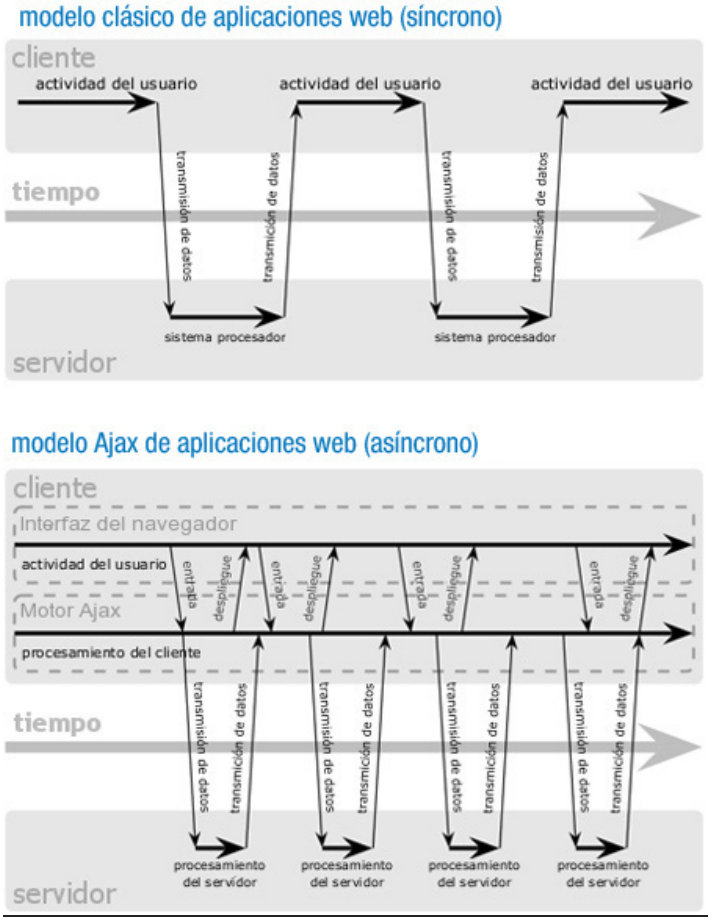
JavaScript, para unir todos los componentes anteriores.

OBJETIVO: poder mostrar la información de forma dinámica e interactiva.

PROBLEMA: Los modelos tradicionales de aplicaciones Web se centran en que casi todos eventos que el usuario produce en la interfaz, disparan solcitudes HTTP al servidor web. Luego el servidor ejecuta el proceso, recogiendo la información y realizando las acciones necesarias, para devolver una página al cliente. Ojo: este tipo de aplicaciones no son siempre las mas recomendables, ya que cada vez que se realiza una petición al servidor, el usuario lo único que puede hacer es esperar, hasta recibir todos los datos del servidor. Hasta ese momento final de carga, no se mostrará el resultado, con lo que el usuario no podrá interactuar de ninguna manera con el navegador.

SOLUCION: Con AJAX, se intenta evitar, todas esas esperas. El cliente podrá hacer solicitudes al servidor, mientras el navegador sigue mostrando la misma página web, y cuando el navegador reciba una respuesta del servidor, la mostrará al cliente y todo ello sin recargar o cambiar de página.





raveiga (CC BY-SA)

AJAX es utilizado por muchas empresas y productos hoy en día. Por ejemplo, Google utiliza AJAX en aplicaciones como Gmail, Google Maps.., así como Flickr, Amazon, etc.

Son muchas las razones para usar AJAX:

- Está basado en estándares abiertos.
- Su usabilidad.
- Válido en cualquier plataforma y navegador.
- Beneficios que aporta a las aplicaciones web.
- Es la base de la web 2.0.
- Es independiente del tipo de tecnología de servidor utilizada.
- Mejora la estética de la web.

7.1.2 Promesas: Async y Await

Este patrón es la forma estándar de gestionar operaciones asíncronas hoy en día. Su objetivo es simplificar la lectura del código, eliminando el "ruido" visual de los bloques .then() y permitiendo que la lógica fluya de forma vertical, casi como si fuera código tradicional.

Función async (El contenedor)

Se usa como prefijo en cualquier función para indicar que operará de forma asíncrona.

Al marcar una función como async, garantizas que siempre retornará una Promesa. Si devuelves un dato (como un número o texto), JavaScript se encarga de empaquetarlo automáticamente en una promesa cumplida.

Operador await (La espera inteligente)

Se coloca antes de una llamada que devuelve una promesa. Le dice al motor de JavaScript: "Detente aquí hasta que esta tarea termine".

Regla de uso: Es obligatorio que esté dentro de una función declarada con async.

Ventaja clave: A diferencia de los métodos antiguos, await no congela la interfaz del usuario; solo suspende la ejecución interna de esa función, permitiendo que el resto de la aplicación siga respondiendo.

7.1.3 API, Métodos y Propiedades del objeto XMLHttpRequest:

- ☑ abort(): Cancela la petición actual.
- ☑ getAllResponseHeaders(): Devuelve la información de cabecera.
- ☑ getResponseHeader(): Devuelve una información de cabecera específica.
- ☑ open(metodo,url,async,usuario,password): Declara la petición y recibe los siguientes parámetros:
    - método: GET o POST (obtener o enviar información al servidor)
    - url: ruta de acceso al fichero
    - async: *true* (asíncrono n-n, La pantalla no será recargada) o *false* (síncrono 1-1)
    - usuario: (opcional) usuario
    - psw: (opcional) password
- ☑ send(): para peticiones GET, envía la petición al servidor
- ☑ send(string): para peticiones POST, envía la petición al servidor
- ☑ setRequestHeader(): añade un par *key/value* a la cabecera de la petición

- ☑ .onreadystatechange: define una función que se llamará cuando la propiedad readyState cambie.
- ☑ .readyState: estado de la petición con los siguientes valores posibles
    - 0: Petición no inicializada.
    - 1: Conexión al servidor establecida.

    - 2: Petición recibida.
    - 3: Procesando petición.
    - 4: Petición finalizada y respuesta lista.
- ☑ .responseText: devuelve la respuesta como un string.
- ☑ .responseXML: devuelve la respuesta como un *XML Data*.
- ☑ .status:devuelve el número de *status* de la petición.
    - 200: "OK"
    - 403: "Forbidden"
    - 404: "Not Found"
- ☑ .statusText: devuelve el *status* en forma de texto (por ejemplo "Forbidden" o "Not Found")

LA PRIMERA APLICACIÓN: CÓDIGO FUENTE

La aplicación AJAX completa más sencilla consiste en una adaptación del clásico "*Hola Mundo*". En este caso, una aplicación JavaScript descarga un archivo del servidor y muestra su contenido sin necesidad de recargar la página. (localhost/holamundo.txt)

```
<!DOCTYPE html>
<html>
<head>
  <title>Hola Mundo con Fetch</title>

  <script type="text/javascript">
    function descargaArchivo() {
      fetch('http://localhost/holamundo.txt')
        .then(response => {
          // Verificamos si la petición fue exitosa (status 200)
          if (!response.ok) {
            throw new Error('Archivo no encontrado');
          }
          // Extraemos el contenido como texto plano
          return response.text();
        })
        .then(contenido => {
          alert(contenido);
        })
        .catch(error => {
          console.error('Error al descargar:', error);
        });
    }
     // Ejecutar al cargar la ventana
    window.onload = descargaArchivo;
  </script>
</head>
<body>
  <h1>Resultado de la petición en el alert</h1>
</body>
</html>
```

En el ejemplo anterior, cuando se carga la página se ejecuta el método JavaScript que muestra el contenido de un archivo llamado `holamundo.txt` que se encuentra en el servidor. La clave del código anterior es que ***la petición HTTP y la descarga de los contenidos del archivo se realizan sin necesidad de recargar la página***.

AJAX no es un lenguaje de programación. Se trata de una técnica de uso en el lado cliente para transferencia de información al servidor. AJAX corresponde con el acrónimo

‘Asynchronous JavaScript And XML’. Su uso se basa en el objeto XMLHttpRequest y fue desarrollado por Alex Hopmann y su equipo mientras trabajaban en Microsoft en el nuevo MS Exchange 2000.

https://web.archive.org/web/20070623125327/http://www.alexhopmann.com/xmlhttp.htm

La idea surgio ante la necesidad de transferir datos al servidor sin necesidad de tener que enviar continuamente los formularios de datos. En un solo fin de semana Hopmann desarrollo la primera versión de XMLHTTP y consigio que se incluyera en el navegador Internet Explorer justo a tiempo del lanzamiento de su última BETA. Ajax utiliza el objeto XMLHttpRequest para comunicarse con el servidor y permite trasferir información usando varios formatos ademas de XML, como por ejemplo JSON, HTML o ficheros de texto. Su caracteristica principal es su ejecución asincrona, lo que permite continuar ejecutando acciones mientras se transfieren los datos y por ende sin necesidad de recargar la página.

Su uso ha permitido un cambio en el paradigma actual de desarrollo Web y permitió que las aplicaciones Web se acercaran considerablemente en cuanto a interfaz de usuario a las aplicaciones nativas de escritorio.

```
// Ejemplo de envío de datos (POST) con Fetch API
const nuevoPost = {
  titulo: 'Mi viaje a Zaragoza',
  contenido: 'Fue una experiencia increíble.',
  userId: 1 };

fetch('https://jsonplaceholder.typicode.com', {
  method: 'POST', // Especificamos el método
  body: JSON.stringify(nuevoPost), // Convertimos el objeto JS a una cadena JSON
  headers: {
    'Content-type': 'application/json; charset=UTF-8', // Indicamos el tipo de contenido
  },
})
  .then(response => {
    if (response.ok) {
      return response.json(); // Si el servidor creó el recurso, nos devuelve la respuesta
    }
    throw new Error('No se pudo crear el recurso');
  })
  .then(data => {
    console.log('Éxito, objeto creado:', data);
    // Mostramos un mensaje de confirmación al usuario
    document.getElementById('resultado').innerText = `Post creado con ID: ${data.id}`;
  })
  .catch(error => {
    console.error('Error al enviar datos:', error);
  });
```

7.1.4 Ajax y Json

Ademas de poder leer ficheros de texto plano o XML, AJAX nos va a permitir recibir datos en formato JSON.

Si no está familiarizado con la sintaxis JSON puede ver un tutorial aqui: https://www.w3schools.com/js/js_json_syntax.asp

En un resumen diremos que los ficheros JSON son prácticamente sintácticamente equivalentes a un objeto de JS. Sin embargo hay algunas diferencias. Una de las restricciones más aparentes es que las key en JSON serán obligatoriamente string. Otra es que, a diferencia de lo que ocurre en JS, no se permite que los values sean de tipo date, function ni undefined. Además los string en los value deberan ir acotados por comillas dobles.

En cuanto a la forma que tiene AJAX de recuperar datos contenidos en un fichero de tipo JSON es muy similar a las vistas con anterioridad.

Config.json

```
{"nombreUsuario": "JuanPérez",
  "tema": "oscuro",
  "idioma": "es-ES",
  "notificaciones": true}
```

```
  async function gestionarConfiguracion() {
  try {
    // 1. Obtener el archivo real
    const respuesta = await fetch('config.json');
    const datosCargados = await respuesta.json(); // Fetch ya hace el JSON.parse() internamente

    // 2. Manipular los datos
    datosCargados.ultimaConexion = new Date().toLocaleDateString();
    datosCargados.visitas = 1;

    // 3. Convertir a string para guardarlo
    // Las cookies solo guardan texto plano
    const jsonParaCookie = JSON.stringify(datosCargados);

    // 4. Guardar en las Cookies
    // Guardamos con una expiración de 7 días
    const diasExpiracion = 7;
    const fecha = new Date();
    fecha.setTime(fecha.getTime() + (diasExpiracion * 24 * 60 * 60 * 1000));

    document.cookie = `preferencias_usuario=${encodeURIComponent(jsonParaCookie)};
    expires=${fecha.toUTCString()}; path=/`;

    console.log("¡Datos guardados en la cookie con éxito!");
    mostrarCookie();}
    catch (error) {
    console.error("Error al procesar el archivo:", error);}}

// Función extra para leer la cookie y volverla a convertir en objeto
function mostrarCookie() {
    const cookies = document.cookie.split(';');
    const miCookie = cookies.find(c => c.trim().startsWith('preferencias_usuario='));

    if (miCookie) {
        const valorCifrado = miCookie.split('=')[1];
        const objetoRecuperado = JSON.parse(decodeURIComponent(valorCifrado));
        console.log("Objeto recuperado desde la Cookie:", objetoRecuperado);}}
```

```
gestionarConfiguracion();
```

7.2. Bibliografía

- ☑ Introducción a JavaScript
- ☑ Fundamentos básicos de Ajax
- ☑ Fundamentos básicos de Json
- ☑ Fundamentos Arbol Dom con JavaScript
- ☑ Almacenamiento local y comunicación asincrona
- ☑ Arbol Dom Formulario Tablas y Eventos con Javascript

7.3. Agradecimientos y reconocimiento

- ☑ La mayoría de lo que hay escrito aquí está mejor explicado en multitud de tutoriales en Internet.

- ☑ Las guías básicas para realizar este material están en la bibliografía de referencia. Cuando se hagan citas literales o ejemplos se indicará su autor.

- ☑ Desear a todo el alumnado, mucha suerte en la nueva etapa que vais a comenzar y el mayor de los éxitos profesionales y personales.

- ☑ Todo un placer compartir conocimientos y experiencias con cada uno de vosotros y vosotras a lo largo de todo el curso. Ya podemos decir que pertenecéis al mundo Front-End aka Vanila JavaScript y espero que sigáis desarrollando aplicaciones con la misma pasión con la que hemos trabajado en clase.

Atentamente para mis amigos y amigas de JS


```
const despedida = () => "Un saludo: Roberto";
```